FAST FORWARD

АКОС

Лекция 5
Язык C. Часть 2

FAST FORWARD

Язык C

- Низкоуровневый компилируемый
- С прямым доступом к памяти
- Предсказуемая компиляция
- Используется для ОС, драйверов, системного ПО

- UB творит беды

ASCII

127 символов

Часть непечатные

dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch
0	0	00	**NUL** (null)	32	40	20	(space)	64	100	40	@	96	140	60	`
1	1	01	**SOH** (start of header)	33	41	21	!	65	101	41	A	97	141	61	a
2	2	02	**STX** (start of text)	34	42	22	"	66	102	42	B	98	142	62	b
3	3	03	**ETX** (end of text)	35	43	23	#	67	103	43	C	99	143	63	c
4	4	04	**EOT** (end of transmission)	36	44	24	$	68	104	44	D	100	144	64	d
5	5	05	**ENQ** (enquiry)	37	45	25	%	69	105	45	E	101	145	65	e
6	6	06	**ACK** (acknowledge)	38	46	26	&	70	106	46	F	102	146	66	f
7	7	07	**BEL** (bell)	39	47	27	'	71	107	47	G	103	147	67	g
8	10	08	**BS** (backspace)	40	50	28	(	72	110	48	H	104	150	68	h
9	11	09	**HT** (horizontal tab)	41	51	29	)	73	111	49	I	105	151	69	i
10	12	0a	**LF** (line feed - new line)	42	52	2a	*	74	112	4a	J	106	152	6a	j
11	13	0b	**VT** (vertical tab)	43	53	2b	+	75	113	4b	K	107	153	6b	k
12	14	0c	**FF** (form feed - new page)	44	54	2c	,	76	114	4c	L	108	154	6c	l
13	15	0d	**CR** (carriage return)	45	55	2d	-	77	115	4d	M	109	155	6d	m
14	16	0e	**SO** (shift out)	46	56	2e	.	78	116	4e	N	110	156	6e	n
15	17	0f	**SI** (shift in)	47	57	2f	/	79	117	4f	O	111	157	6f	o
16	20	10	**DLE** (data link escape)	48	60	30	0	80	120	50	P	112	160	70	p
17	21	11	**DC1** (device control 1)	49	61	31	1	81	121	51	Q	113	161	71	q
18	22	12	**DC2** (device control 2)	50	62	32	2	82	122	52	R	114	162	72	r
19	23	13	**DC3** (device control 3)	51	63	33	3	83	123	53	S	115	163	73	s
20	24	14	**DC4** (device control 4)	52	64	34	4	84	124	54	T	116	164	74	t
21	25	15	**NAK** (negative acknowledge)	53	65	35	5	85	125	55	U	117	165	75	u
22	26	16	**SYN** (synchronous idle)	54	66	36	6	86	126	56	V	118	166	76	v
23	27	17	**ETB** (end of transmission block)	55	67	37	7	87	127	57	W	119	167	77	w
24	30	18	**CAN** (cancel)	56	70	38	8	88	130	58	X	120	170	78	x
25	31	19	**EM** (end of medium)	57	71	39	9	89	131	59	Y	121	171	79	y
26	32	1a	**SUB** (substitute)	58	72	3a	:	90	132	5a	Z	122	172	7a	z
27	33	1b	**ESC** (escape)	59	73	3b	;	91	133	5b	[	123	173	7b	{
28	34	1c	**FS** (file separator)	60	74	3c	<	92	134	5c	\	124	174	7c	|
29	35	1d	**GS** (group separator)	61	75	3d	=	93	135	5d	]	125	175	7d	}
30	36	1e	**RS** (record separator)	62	76	3e	>	94	136	5e	^	126	176	7e	~
31	37	1f	**US** (unit separator)	63	77	3f	?	95	137	5f	_	127	177	7f	**DEL** (delete)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8x	Ђ	Ѓ	‚	ѓ	„	…	†	‡	€	‰	Љ	‹	Њ	Ќ	Ћ	Џ
9x	ђ	‘	’	“	”	•	–	—		™	љ	›	њ	ќ	ћ	џ
Ax	NBSP	Ў	ў	Ј	¤	Ґ	¦	§	Ё	©	Є	«	¬	SHY	®	Ї
Bx	°	±	І	і	ґ	µ	¶	·	ё	№	є	»	ј	Ѕ	ѕ	ї
Cx	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
Dx	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
Ex	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
Fx	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я

Windows 1251

Используем старшие 128 символов.

Каждый язык - своя кодировка.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	¤/$[a] 00A4	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8x																
9x																
Ax																
Bx																
Cx	ю 044E	а 0430	б 0431	ц 0446	д 0434	е 0435	ф 0444	г 0433	х 0445	и 0438	й 0439	к 043A	л 043B	м 043C	н 043D	о 043E
Dx	п 043F	я 044F	р 0440	с 0441	т 0442	у 0443	ж 0436	в 0432	ь 044C	ы 044B	з 0437	ш 0448	э 044D	щ 0449	ч 0447	ъ 044A
Ex	Ю 042E	А 0410	Б 0411	Ц 0426	Д 0414	Е 0415	Ф 0424	Г 0413	Х 0425	И 0418	Й 0419	К 041A	Л 041B	М 041C	Н 041D	О 041E
Fx	П 041F	Я 042F	Р 0420	С 0421	Т 0422	У 0423	Ж 0416	В 0412	Ь 042C	Ы 042B	З 0417	Ш 0428	Э 042D	Щ 0429	Ч 0427	

KOI-8

Читаемо если открыть ASCII only редактором

Привет -> pRIVET

FAST FORWARD

Unicode

Единая таблица всех символов мира

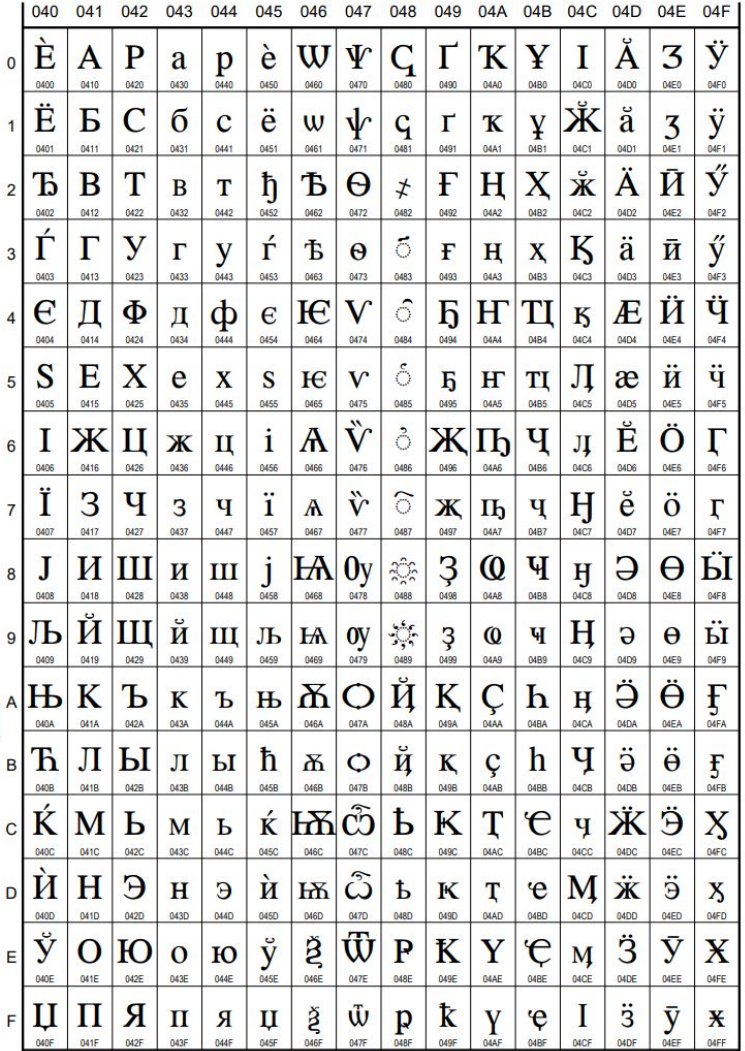
Содержит звуки, эмодзи, мертвые языки, Эсперанто

Максимум 1 114 112

Сейчас ~160 000

Первые 127 как в ASCII

Кириллица



1. Cyrillic: U+0400–U+04FF
2. Cyrillic Supplement: U+0500–U+052F
3. Cyrillic Extended-A: U+2DE0–U+2DFF
4. Cyrillic Extended-B: U+A640–U+A69F
5. Cyrillic Extended-C: U+1C80–U+1C8F
6. Cyrillic Extended-D: U+1E030–U+1E08F

ё - отдельно

FAST FORWARD

UTF-16 / UTF-32

wchar_t - 4 байта unix, 2 байта win

std::wstring

Значение - номер в таблице Unicode

+ Символ по индексу за O(1)
+ Размер строки по размеру в байтах за O(1)

- Для a-z A-Z 0-9 в два раза больше места
- На windows не все символы

UTF-8

0xxxxxxx — символ ASCII

110xxxxx 10xxxxxx

1110xxxx 10xxxxxx 10xxxxxx

11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

+ Переменная длина
+ Компактно - a-z A-Z 0-9 по 1 байту
+ Дефолтная для современных unix и web

- Переменная длина
- Размер и индексация за O(N)
- Китайские иероглифы не влезают в 2 байта

FAST FORWARD

Сдвиги

```
7 << 2                  // 28
7 << -1                 // UB
7 << 30                 // на нашей платформе UB
-1 >> 1                 // implementation defined
0xABCDEFFFu << 4        // 0xBCDEFFF0u
0 << 32                 // на нашей платформе UB
```

FAST
FORWARD

Сдвиги

```
_Bool is_ascii(char b) {
    // return (b >> 7) == 0;  // nope, could be signed char
    return ((unsigned char) b >> 7) == 0;
}
_Bool is_continuation(unsigned char b) {
    // return (b >> 6) == 0b10; // valid in C++14
    return (b >> 6) == 2;
}
_Bool is_2_byte_start(unsigned char b) {
    return (b >> 5) == 6; // 0b110
}
```

FAST FORWARD

Маски

```
_Bool is_2_byte_start(unsigned char b) {
    return (b & 0xE0) == 0xC0;  // 0b1110'0000, 0b1100'0000
}
```

FAST FORWARD

Bit vs Byte

CHAR_BIT бит в **char**

Разные инстансы тогда и только тогда когда **разные указатели**

```
int *p;
int *q;
if (p == q)  // Один и тот же объект
```

FAST FORWARD

<stdbool.h>

_Bool - deprecated since C23

bool

An object declared as type _Bool is large enough to store the values 0 and 1. (WG14 N1570 6.2.5.3)

Содержит true или false

FAST
FORWARD

<stdbool.h>

```
sizeof(bool)     == 1
sizeof(bool[8]) == 8  // восемь байт


(bool)0.5 == true
(int)0.5    == false
```

FAST FORWARD

std::vector<bool>

C++

```cpp
#include <vector>

auto func() {
    std::vector<bool> v(10, false);
    v[5] = true;
    return v;
}
```

x86-64 gcc 15.2 -O0

```asm
        call    std::__new_allocator<bool>::~__new_allocator(
        nop
        mov     rax, QWORD PTR [rbp-72]
        mov     esi, 5
        mov     rdi, rax
        call    std::vector<bool, std::allocator<bool>>::opera
        mov     QWORD PTR [rbp-48], rax
        mov     QWORD PTR [rbp-40], rdx
        lea     rax, [rbp-48]
        mov     esi, 1
        mov     rdi, rax
        call    std::_Bit_reference::operator=(bool)
        jmp     .L33
        mov     rbx, rax
        lea     rax, [rbp-50]
```

FAST FORWARD

std::vector<bool>

C++

```cpp
#include <vector>

auto func() {
    std::vector<bool> v(10, false);
    v[5] = true;
    return v;
}
```

x86-64 gcc 15.2 -O2

```asm
func():
        push    rbx
        mov     rbx, rdi
        mov     edi, 8
        sub     rsp, 16
        call    operator new(unsigned long)
        mov     DWORD PTR [rbx+8], 0
        lea     rdx, [rax+8]
        mov     QWORD PTR [rbx], rax
        mov     QWORD PTR [rbx+16], rax
        mov     QWORD PTR [rax], 32
        mov     rax, rbx
        mov     QWORD PTR [rbx+32], rdx
        mov     DWORD PTR [rbx+24], 10
        add     rsp, 16
        pop     rbx
        ret
```

FAST FORWARD

std::vector<bool>

C++

```cpp
#include <vector>

auto func(std::vector<bool> v, int idx) {
    v[idx] = true;
}
```

x86-64 gcc 15.2 -O2

```asm
func(std::vector<bool, std::allocator<bool>>, int):
        mov     rdi, QWORD PTR [rdi]
        movsx   rax, esi
        mov     edx, 1
        mov     ecx, esi
        shr     rax, 6
        sal     rdx, cl
        or      QWORD PTR [rdi+rax*8], rdx
        ret
```

Union

```
union U {
    uint32_t u;
    char bytes[4];
}
```

alignof = max(alignof(elems…))

sizeof = max(sizeof(elems…))

FAST FORWARD

Union

```
union U {
    uint32_t u;
    char bytes[4];
}

union U u;
u.u = 10;
u.bytes[0] == ?;
```

C - reinterpretation of object representation

C++ - UB

FAST FORWARD

Битовые поля

```
struct u8char {
    uint8_t sign_bit : 1;
    uint8_t tail_bits : 7;
}
sizeof(struct u8char) == 1
```

Порядок бит - Implementation Defined

Если место в “unit” есть - кладем

Иначе - Implementation Defined

FAST
FORWARD

Битовые поля

```
union char_breaker {
    uint8_t number;
    struct u8char {
        uint8_t sign_bit : 1;
        uint8_t tail_bits : 7;
    }
}
union char_breaker cb = {.number = 'x'};
cb.fields.sign_bit;
// implementation defined if this is the sign bit :-(
```

FAST FORWARD

Typedef

```
typedef int my_int_32_t;
typedef struct foobar {
    ...
} foobar;

foobar f;
```

FAST
FORWARD

const

```
const char *s1, *s2;
const int x = 70;
const char * const msg = "hi";

y = *((int*)&x);  // OK
*((int*)&x) = 5;  // UB
```

FAST FORWARD

const

C

```c
#include <stdio.h>

int main() {
    const int x = 70;
    *(int*)&x = 5;
    printf("%d\n", x);
}
```

x86-64 gcc 15.2 -O0

```asm
.LC0:
        .string "%d\n"
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     DWORD PTR [rbp-4], 70
        lea     rax, [rbp-4]
        mov     DWORD PTR [rax], 5
        mov     eax, DWORD PTR [rbp-4]
        mov     esi, eax
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        leave
        ret
```

FAST FORWARD

const

C

```c
#include <stdio.h>

int main() {
    const int x = 70;
    *(int*)&x = 5;
    printf("%d\n", x);
}
```

x86-64 gcc 15.2 -O2

```asm
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 70
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

FAST FORWARD

const

C

```c
#include <stdio.h>

int main() {
    const int x = 70;
    *(int*)&x = 5;
    printf("%d\n", x);
}
```

x86-64 gcc 5.4 -O2

```asm
.LC0:
        .string "%d\n"
main:
        sub     rsp, 8
        mov     esi, 5
        mov     edi, OFFSET FLAT:.LC0
        xor     eax, eax
        call    printf
        xor     eax, eax
        add     rsp, 8
        ret
```

FAST FORWARD

volatile

volatile int x;

May be modified in ways unknown to the implementation or have other unknown side effects

Это **НЕ** атомарность, **НЕ** барьер и **НЕ** thread-safe.

Применяется:

- memory-mapped I/O
- сигналы (только для типа sig_atomic_t)

FAST FORWARD

Указатели на функции

```
int strcmp(const char *a, const char *b);

int *comparator(const char *a, const char *b) = strcmp;

int (*my_comparator)(const char *a, const char *b) = strcmp;
```

FAST FORWARD

Указатели на функции

```
typedef int (*comparator_t)(const char *, const char *);

comparator_t comparator = strcmp;

int result1 = (*comparator)("Roger", "David");
int result2 = comparator("Nick", "Rick");

qsort(..., comparator, ...);
```

FAST FORWARD

Перегрузка оператора - только в C++

```
struct Vec {
    int x, y;

    Vec operator+(const Vec& other) const {
        return {x + other.x, y + other.y};
    }
};
```

FAST FORWARD

Указатели на функции

```
void func(void);

void *(*(fn_t)(int x))(void) {
    return func;
}
```

FAST FORWARD

Fixed-size print

```
uint32_t var;
printf("%...", var);    // ???

printf("%d", var);      // fails on Win16
printf("%ld", var);     // fails on 64-bit systems

scanf("%...", &var); // even worse
```

FAST FORWARD

Fixed-size print

```
#include <inttypes.h>

uint32_t var;
uintptr_t ptr;

printf("%" PRId32 "\n", var);   // "%" "d" "\n"
scanf("%" SCNi32 "\n", &var);
scanf("%" SCNiPTR "\n", &ptr);
```

FAST
FORWARD

Fixed-size

```
char buf[100];
// scanf("%s", buf); <- banned
scanf("%99s", buf);
...
snprintf(buf, sizeof(buf), "%d bottles of beer on the wall", 50)
...
fgets(buf, sizeof(buf), stdin);
// gets(buf);  <- banned from C11
```

FAST FORWARD

Dynamic memory

```
char *buf = NULL;
scanf("%ms", &buf);  // POSIX
free(buf);
```

FAST FORWARD

GCC builtins overflow

```
bool __builtin_add_overflow (type1 a, type2 b, type3 *res);
bool __builtin_sub_overflow (type1 a, type2 b, type3 *res);
bool __builtin_mul_overflow (type1 a, type2 b, type3 *res);

char a = 100, b = 200, c;
int d;
__builtin_add_overflow(a, b, &c);
// -> true == overflow
__builtin_add_overflow(a, b, &d);
// -> false == no overflow
```

FAST FORWARD

GCC builtins bits

```
int __builtin_ffs(int x);                  // 1 + индекс младшего 1-бита, или 0
int __builtin_clz(unsigned x);             // число ведущих нулей, UB на x == 0
int __builtin_ctz(unsigned x);             // число замыкающих нулей, UB на x == 0
int __builtin_clrsb(int x);                // число лишних знаковых битов
int __builtin_popcount(unsigned x);        // число единичных битов
int __builtin_parity(unsigned x);          // popcount(x) % 2
```

FAST
FORWARD

goto

```
int func(int x) {
    if (x % 2 == 0) {
        goto error
    }
    …

    error:
        // process error
}
```

FAST FORWARD

GOTOphobia

- Error/exception handling & cleanup
  - goto-less alternative 1: nested ifs
  - goto-less alternative 2: if not then clean
  - goto-less alternative 3: flags
  - goto-less alternative 3.5: so-far-ok flag
  - goto-less alternative 4: functions
  - goto-less alternative 5: abuse of loops
- Restart/retry
  - goto-less alternative: loop
- Common code in switch statement
  - goto-less alternative 1: functions
  - goto-less alternative 2: ifs
  - goto-less alternative 3: interlacing if (0)
  - goto-less alternative: capturing lambda

Heap - куча

Стек - непрерывный

Куча - “беспорядочная”

FAST
FORWARD

libc

```
#include <stdlib.h>

// C11, POSIX
void *malloc(size_t size);  // Если ошибка то NULL
void free(void *ptr);       // NULL можно но не везде
```

FAST
FORWARD

Simple

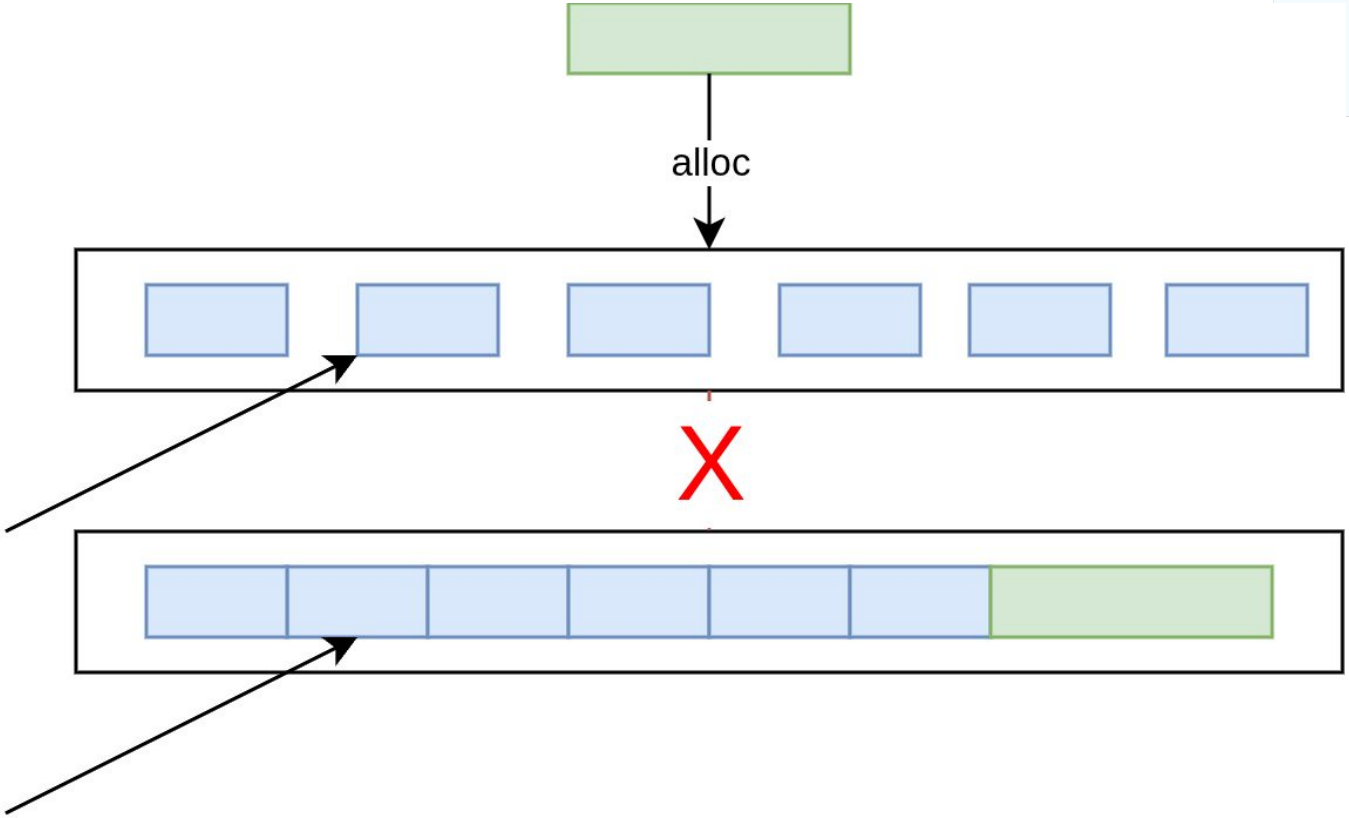
```
char *buf = malloc(buf_size);
// type conversion is automatic
// buf == NULL -> allocation error
// otherwise, contents of buf undefined here

strcpy(buf, "arc de Triomphe");
buf[0] = 'A';

free(buf);
```

FAST
FORWARD

UB

```
char * ptr = malloc(10);
free(ptr); free(ptr);  // double free

*ptr = 0;              // use after free

free(ptr + 1);         // invalid ptr

ptr[-1]; ptr[11];      // buffer overflow
```

OOM



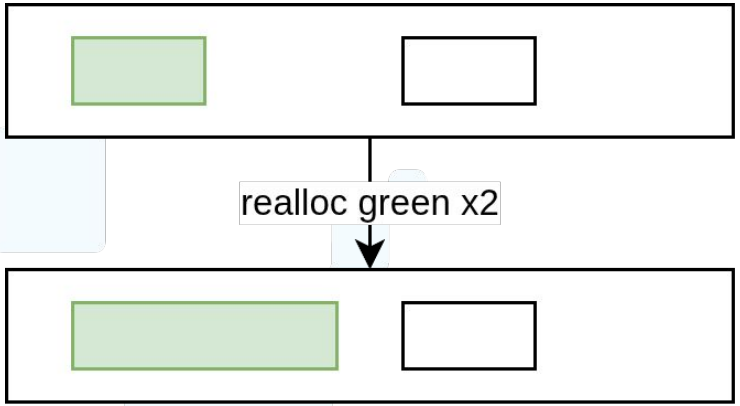
FAST
FORWARD

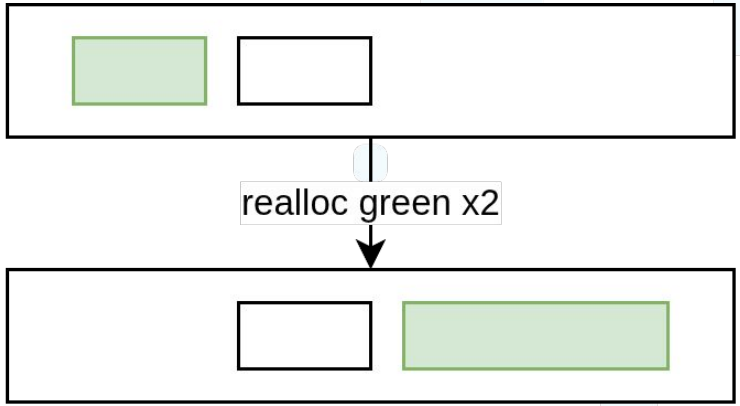
libc

```
// C11, POSIX
void *malloc(size_t size);
void free(void *ptr);

void *calloc(size_t nmemb, size_t size); // Нулевые
void *realloc(void *ptr, size_t size);        // Поменять размер

// nonstandard
void *reallocarray(void *ptr, size_t nmemb, size_t size);
```

FAST FORWARD

realloc





new_ptr = realloc(ptr, n) // Если места недостаточно - NULL

malloc(n) == realloc(NULL, n);

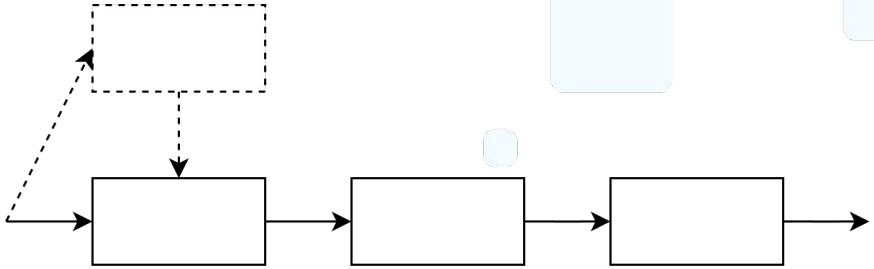
free(ptr) == realloc(ptr, 0);

FAST FORWARD

calloc

```
int *array = calloc(30, sizeof(*array));

int *array = malloc(30 * sizeof(*array));
// check for NULL
memset(array, 0, 30 * sizeof(*array));
```

list



```
struct List {
    int data;
    struct List *next;
};
void insert_front(struct List ** head, int key) {
    new_head = calloc(1, sizeof(**head));
    new_head->data = key;
    new_head->next = *head;
    *head = new_head;
}
```

FAST
FORWARD

void*

void * <-> any *

char * <-> any *

FAST FORWARD

void*

6.3.2.3 Pointers

1 A pointer to **`void`** may be converted to or from a pointer to any object type. A pointer to any object type may be converted to a pointer to **`void`** and back again; the result shall compare equal to the original pointer.

7 A pointer to an object type may be converted to a pointer to a different object type. If the resulting pointer is not correctly aligned[68)] for the referenced type, the behavior is undefined. Otherwise, when converted back again, the result shall compare equal to the original pointer. When a pointer to an object is converted to a pointer to a character type, the result points to the lowest addressed byte of the object. Successive increments of the result, up to the size of the object, yield pointers to the remaining bytes of the object.

8 A pointer to a function of one type may be converted to a pointer to a function of another type and back again; the result shall compare equal to the original pointer. If a converted pointer is used to call a function whose type is not compatible with the referenced type, the behavior is undefined.

FAST FORWARD

void* ?

```
void * <-> any *
char * <-> any *

int * -> void *              // YES
int * -> long *              // NO
int * -> void * -> long *    // YES???
```

FAST FORWARD

Strict aliasing

```
2
3
4
5  void adds(int  *a, int *b) {
6      *a += *b;
7      *a += *b;
8  }
9
```

```
1  adds:
2          movl    8(%esp), %eax
3          movl    4(%esp), %ecx
4          movl    (%ecx), %edx
5          addl    (%eax), %edx
6          movl    %edx, (%ecx)
7          addl    (%eax), %edx
8          movl    %edx, (%ecx)
9          retl
```

FAST FORWARD

Strict aliasing

```c
void adds(int *restrict a,
          int *restrict b) {
    *a += *b;
    *a += *b;
}
```

```
adds:
        movl    4(%esp), %eax
        movl    8(%esp), %ecx
        movl    (%ecx), %ecx
        addl    %ecx, %ecx
        addl    %ecx, (%eax)
        retl
```

FAST FORWARD

Strict aliasing

7 An object shall have its stored value accessed only by an lvalue expression that has one of the following types:[88)]

— a type compatible with the effective type of the object,

— a qualified version of a type compatible with the effective type of the object,

— a type that is the signed or unsigned type corresponding to the effective type of the object,

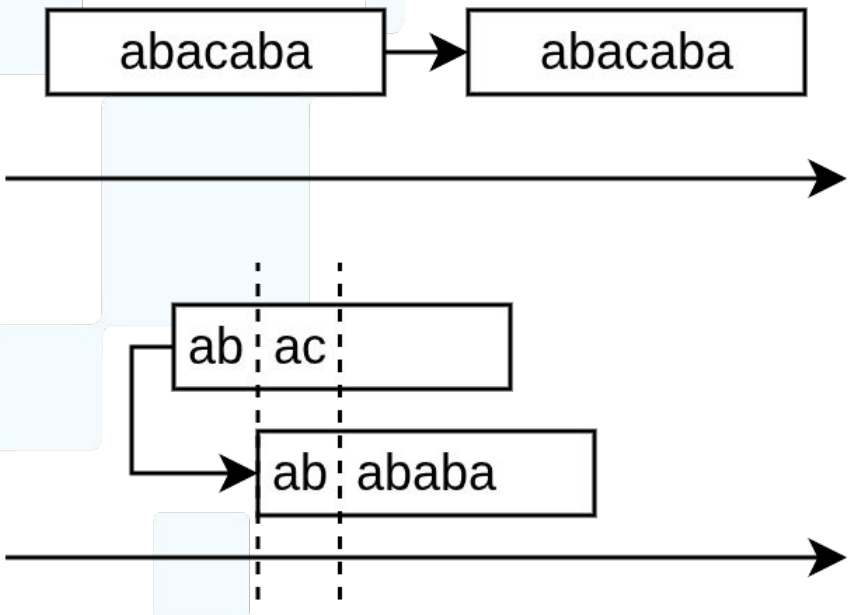
— a type that is the signed or unsigned type corresponding to a qualified version of the effective type of the object,

— an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or

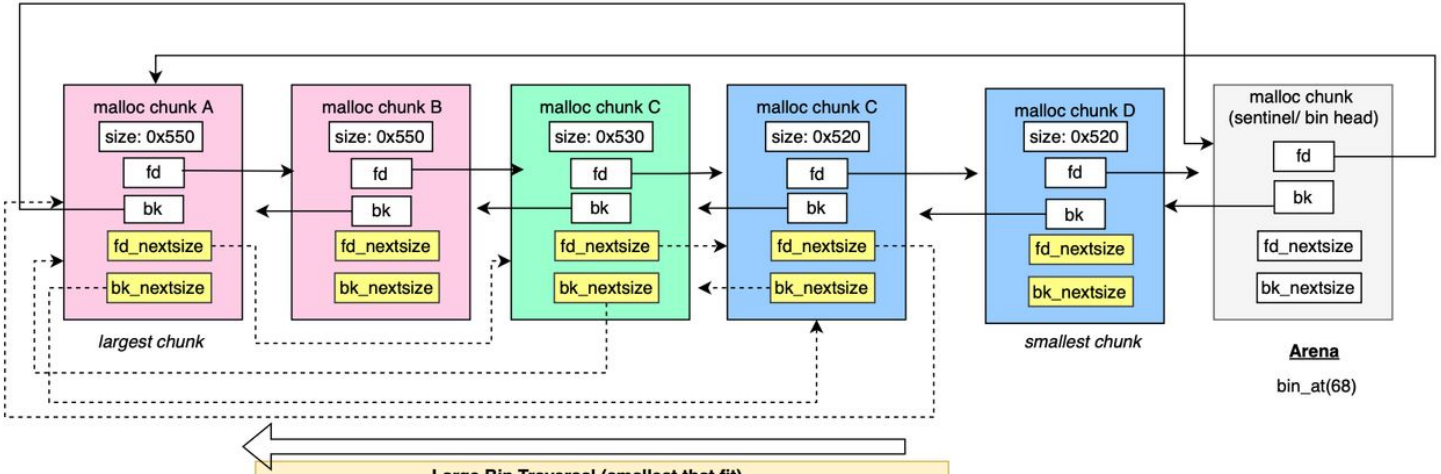
— a character type.

memcpy

```
void memcpy(char * dst, char * src);
```



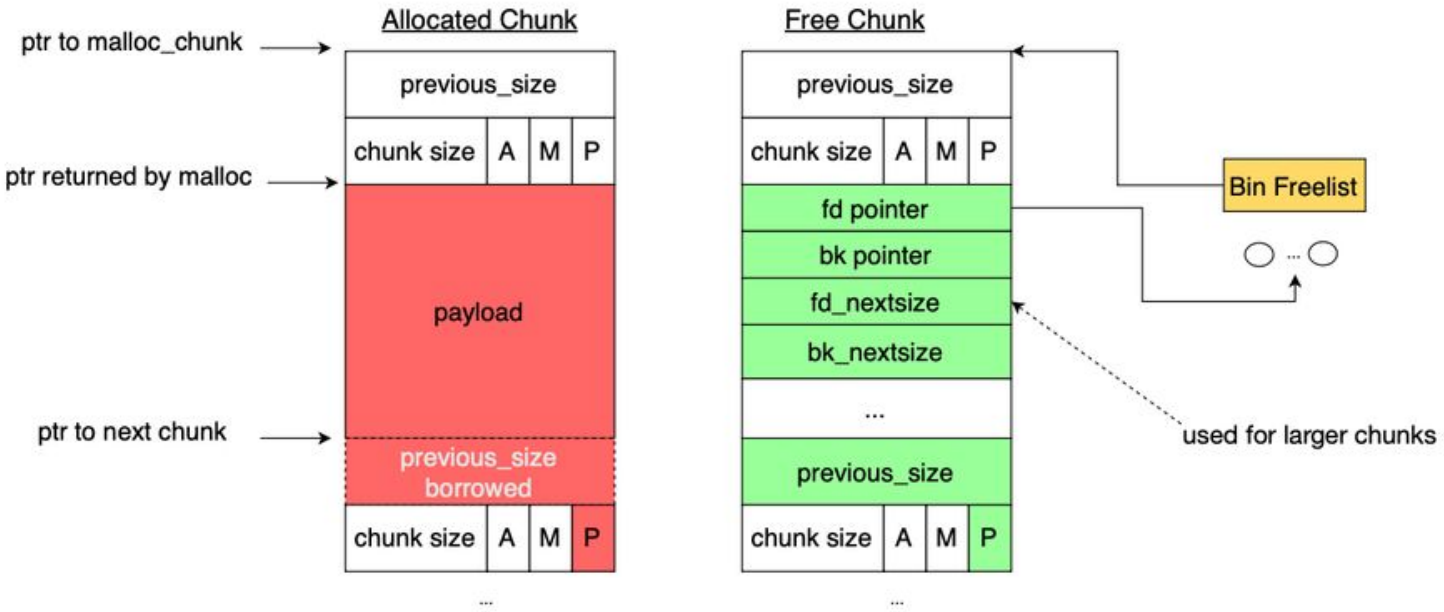
Glibc



Large Bin Traversal (smallest that fit)

```
if ((victim = first (bin)) != bin
&& (unsigned long) chunksize_nomask (victim)
  >= (unsigned long) (nb))
  {
    victim = victim->bk_nextsize;
    while (((unsigned long) (size = chunksize (victim)) <
            (unsigned long) (nb)))
      victim = victim->bk_nextsize;
```

Glibc


Allocated Chunk
ptr to malloc_chunk
previous_size
chunk size
A
M
P
ptr returned by malloc
payload
ptr to next chunk
previous_size borrowed
chunk size
A
M
P
...
Free Chunk
previous_size
chunk size
A
M
P
fd pointer
bk pointer
fd_nextsize
bk_nextsize
...
previous_size
chunk size
A
M
P
...
Bin Freelist
...
used for larger chunks

FAST FORWARD

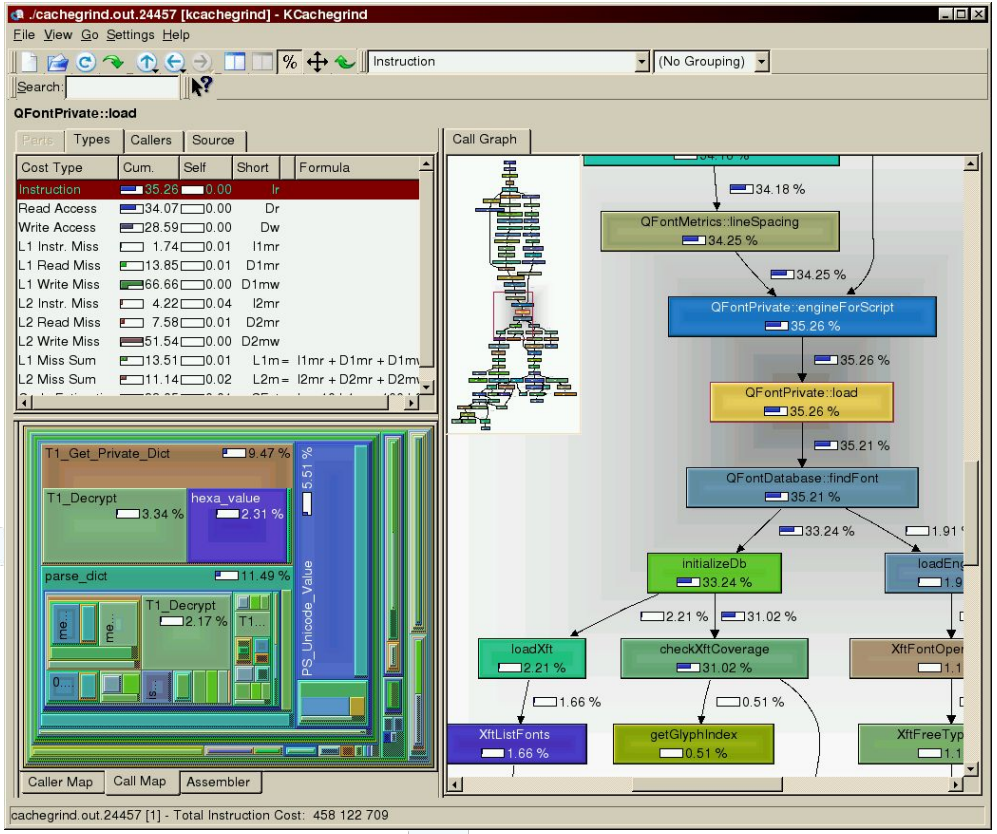
Другие языки

C++'s new uses malloc

Python: malloc + reference counting + GC

Go, Java: garbage collection

Как искать ошибки?



1. -fsanitize=address

   Требует перекомпиляции

2. valgrind

   Медленно работает

FAST FORWARD

АКОС

QCourse.ru 2026